Two-dimensional convolution and pooling

Week 3, Session 8 Thursday, 19:20–20:20

How does a local operation turn an image into feature maps?

By the end of this session

- Compute a 2D convolutional layer.
- Track channels and count parameters.
- Compare max and average pooling.
- Trace a unit back to its input region.

Minutes	Focus
0–15	The 2D window
15–30	Channels and filters
30–45	Pooling and receptive field
45–60	Code and RGB exercise

The ideas from 1D extend along two spatial axes.

Mathematical and Computational Foundations | Francisco R. Villatoro. Main slides 1–20; reference appendices 21–26.

A two-dimensional window

One input channel, one filter, stride 1, no padding, and $b = 0$.

Input X: 5×5

	0	1	2	3	4
0	2	2	0	1	3
1	3	1	2	2	0
2	0	1	3	1	2
3	2	2	1	0	1
4	1	0	2	3	1

Indices start at 0

Kernel W

1	-1
1	-1

Stored order

One local sum

$$2(1) + 2(-1) + 3(1) + 1(-1)$$
$$= 2 - 2 + 3 - 1$$

$$Z_{0,0} = 2$$

Reuse these four weights at every location.

The same 2×2 weights move across rows and columns.

At each position: multiply corresponding entries, sum, and add the bias.

Original synthetic image and kernel; indices start at zero. We hold weights fixed to calculate the forward pass. Locality and sharing: Goodfellow, Bengio & Courville (GBC), Ch. 9, §§9.1–9.2.

The 2D cross-correlation operation

For a $k_H \times k_W$ kernel, stride 1 and no padding:

$$\boxed{Z_{i,j} = b + \sum_{u=0}^{k_H-1} \sum_{v=0}^{k_W-1} W_{u,v}\, X_{i+u,j+v}}$$

i, j: window position. u, v: offset inside the kernel.

First window

$$\begin{bmatrix} 2 & 2 \\ 3 & 1 \end{bmatrix}, \qquad W = \begin{bmatrix} 1 & -1 \\ 1 & -1 \end{bmatrix}$$

$Z_{0,0} = 2 - 2 + 3 - 1 = \boxed{2}$.

One row down, one column right

$$\begin{bmatrix} 1 & 2 \\ 1 & 3 \end{bmatrix}, \qquad \text{same } W$$

$Z_{1,1} = 1 - 2 + 1 - 3 = \boxed{-3}$.

Libraries call this convolution; the kernel is not flipped. PyTorch Conv2d; GBC, §9.1. Valid indices: $0 \le i \le H_{\text{in}} - k_H$, $0 \le j \le W_{\text{in}} - k_W$.

A feature map and its activation

The 5×5 image produces a 4×4 map of local scores.

Preactivation

$$Z = \begin{bmatrix} 2 & 1 & -1 & 0 \\ 1 & -3 & 2 & 1 \\ -1 & -1 & 3 & -2 \\ 1 & -1 & 0 & 1 \end{bmatrix}$$

Pointwise activation

$$H = \mathrm{ReLU}(Z) = \begin{bmatrix} 2 & 1 & 0 & 0 \\ 1 & 0 & 2 & 1 \\ 0 & 0 & 3 & 0 \\ 1 & 0 & 0 & 1 \end{bmatrix}$$

- ReLU changes values and preserves the tensor shape.
- Each location reuses the same four weights and one bias.
- A feature map records *where* this filter responds.

Original calculation. Here the bias has value zero but remains a trainable parameter. GBC, Ch. 6 (ReLU) and Ch. 9 (convolutional layers).

Output height and width

Count valid window starts independently along each spatial axis.

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} + p_T + p_B - k_H}{s_H} \right\rfloor + 1,$$

$$W_{\text{out}} = \left\lfloor \frac{W_{\text{in}} + p_L + p_R - k_W}{s_W} \right\rfloor + 1.$$

p_T, p_B: top and bottom padding

p_L, p_R: left and right padding

s_H: vertical stride

s_W: horizontal stride

Our example: $H_{\text{out}} = W_{\text{out}} = \lfloor (5-2)/1 \rfloor + 1 = 4$.

Height and width may use different sizes, strides and padding.

Dilation 1; positive strides; nonnegative padding; at least one full window must fit the padded input. PyTorch Conv2d. The dilated case is in Appendix 21.

Images with several channels

An RGB image has **two spatial axes** and **three components per pixel**.

$$\text{one pixel } (i,j): \quad (X_{R,i,j},\ X_{G,i,j},\ X_{B,i,j}).$$

Input channels

RGB gives $C_{\text{in}} = 3$.

All three planes refer to the same pixel grid.

In later layers, channels are learned features.

A spatial filter

A 3×3 filter reads a patch from *every* input channel.

For RGB, that is $3 \times 3 \times 3 = 27$ values at each location.

Conv2D slides in height and width; it sums over channels.

We use ordinary dense channel mixing, `groups=1`. The channel axis is not a third sliding spatial axis. GBC, §9.5; PyTorch Conv2d.

A filter combines all input channels

For output channel o, sum channel contributions and add its bias once:

$$Z_{n,o,i,j} = b_o + \sum_{c=0}^{C_{\text{in}}-1} \text{corr2D}(X_{n,c}, W_{o,c})_{i,j}.$$

Two-channel example, one output location: $n = 0, o = 0, i = 0, j = 1$.

Channel 0 patch and kernel slice

$$\begin{bmatrix} 2 & 0 \\ 1 & 2 \end{bmatrix}, \quad \begin{bmatrix} 1 & -1 \\ 1 & -1 \end{bmatrix}$$

Contribution: $2 - 0 + 1 - 2 = 1$.

Channel 1 patch and kernel slice

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Contribution: $1 + 0 + 0 + 1 = 2$.

$$Z_{0,0,0,1} = 1 + 2 + \underbrace{1}_{b_0} = 4$$

Original two-channel example; full arrays in the companion programs. GBC, §9.5. Full indexed operator in Appendix 21.

A bank of filters

Each output channel has its own full filter and its own bias.

Output channel	Slice for input 0	Slice for input 1	Bias
$o = 0$	$\begin{bmatrix} 1 & -1 \\ 1 & -1 \end{bmatrix}$	$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$	1
$o = 1$	$\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$	$\begin{bmatrix} -1 & 0 \\ 0 & -1 \end{bmatrix}$	-2

$$X : (1, 2, 5, 5) \quad \longrightarrow \quad Z : (1, 2, 4, 4) \qquad \text{(NCHW)}.$$

At position $(0, 1)$: the output-channel vector is $(4, 1)$.

Two filters produce two maps; each map combines both input channels.

Original example; stride 1, no padding, `groups=1`. Each displayed coefficient is a separate trainable variable, including those currently equal to zero.

Trainable parameters in Conv2D

For C_{out} filters, each covering C_{in} input channels:

$$\boxed{P = C_{\text{out}}\left(C_{\text{in}} k_H k_W + 1\right)} \qquad \text{with bias.}$$

Contribution	Two-channel, two-filter example
Kernel weights	$C_{\text{out}} C_{\text{in}} k_H k_W = 2 \cdot 2 \cdot 2 \cdot 2 = 16$
Biases	$C_{\text{out}} = 2$
Total	$P = 16 + 2 = \boxed{18}$

- Batch size and image resolution do not enter this count.
- Larger images create more activations and more computation.

groups=1. Without bias, omit $+1$. Count derived from the documented weight and bias shapes: PyTorch Conv2d.

Tensor layouts across libraries

Same operation, different axis order. Here B denotes batch size.

Tensor	PyTorch	Keras / TensorFlow
Input X	(B, C_{in}, H, W)	(B, H, W, C_{in})
Kernel W	$(C_{\text{out}}, C_{\text{in}}, k_H, k_W)$	$(k_H, k_W, C_{\text{in}}, C_{\text{out}})$
Output Z	$(B, C_{\text{out}}, H', W')$	$(B, H', W', C_{\text{out}})$

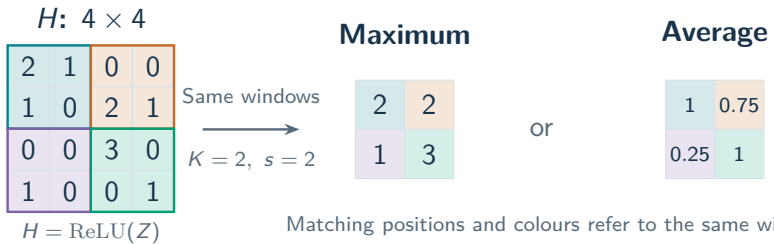
```
# NumPy arrays: NCHW -> NHWC; OIHW -> HWIO
x_nhwc = x_nchw.transpose(0, 2, 3, 1)
w_hwio = w_oihw.transpose(2, 3, 1, 0)
```

Permute the axes to preserve their meaning. A reshape is insufficient.

Keras with TensorFlow backend and explicit `channels_last`; `groups=1`. PyTorch Conv2d; Keras Conv2D; TensorFlow conv2d.

Max pooling and average pooling

Return to $H = \mathrm{ReLU}(Z)$ from slide 4. Use 2×2 windows and stride 2.



Max pooling: strongest response

First window: $\max(2, 1, 1, 0) = 2$.

Average pooling: mean response

First window: $(2 + 1 + 1 + 0)/4 = 1$.

Spatial pooling is performed independently in each channel. No padding in this example. Original arrays; GBC, §9.3; Keras MaxPooling2D.

Pooling shapes and information

For a $q_H \times q_W$ pooling window, no padding and floor sizing:

$$H' = \left\lfloor \frac{H - q_H}{s_H} \right\rfloor + 1, \qquad W' = \left\lfloor \frac{W - q_W}{s_W} \right\rfloor + 1.$$

$$(B, C, 4, 4) \xrightarrow{2\times 2,\ s=2} (B, C, 2, 2).$$

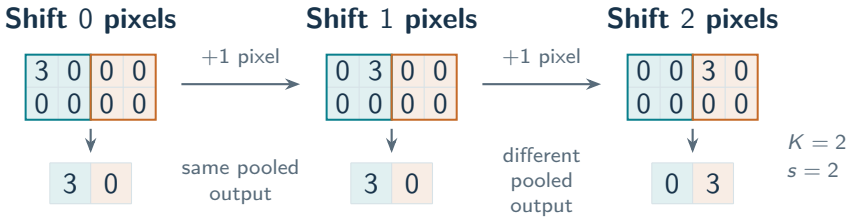
- Pooling preserves batch size and channel count.
- Max and average pooling have **zero trainable parameters**.
- Four values become one: spatial detail is lost.
- Window size and stride are distinct hyperparameters.

A smaller feature map saves computation and changes the information kept.

PyTorch MaxPool2d, `ceil_mode=False`; Keras MaxPooling2D, `padding="valid"`. With a 5×5 input in this configuration, the last row and column are unused.

Pooling and translations

Move a response one pixel at a time; apply max pooling with 2×2, stride 2.



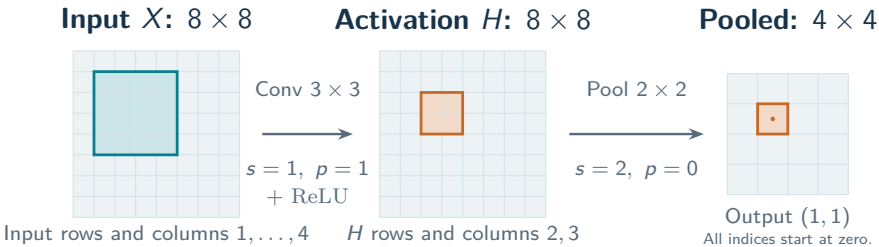
- A move within the same window may preserve the pooled value.
- Crossing a window boundary can change the output map.

Local tolerance does not imply exact translation invariance.

Original counterexample. Away from boundary effects, an input shift by two pixels aligns with one output step here. GBC, §9.3. Strided pooling does not guarantee insensitivity to all small translations.

Receptive field and output spacing

Now use a 3×3 convolution, stride 1, padding 1, then 2×2 pooling.



Receptive field r

Input extent that may influence one output.

Here: a 4×4 region for an interior unit.

Output spacing j, or jump

Distance between adjacent outputs in input coordinates. Here: 2 pixels.

Original dependency diagram; an optional pointwise ReLU preserves these extents. We track geometric potential dependency, not the actual gradient for a particular image. Araujo, Norris & Sim, Distill (2019).

Receptive fields through a stack

Compute one spatial axis at a time. Start with $r_0 = j_0 = 1$.

$$\boxed{r_\ell = r_{\ell-1} + (k_\ell - 1) j_{\ell-1}}, \qquad \boxed{j_\ell = s_\ell j_{\ell-1}}.$$

Operation	k	s	(r, j)	Input extent
Input	–	–	$(1, 1)$	1×1
Conv 3×3, padding 1	3	1	$(3, 1)$	3×3
ReLU	1	1	$(3, 1)$	unchanged
Pool 2×2, no padding	2	2	$(4, 2)$	4×4

Why 4, rather than 6? Adjacent 3×3 input regions overlap.

Dilation 1, sequential local operators. Padding affects alignment and coverage at the image boundary, not this nominal span. Araujo, Norris & Sim (2019); alignment and dilation in Appendix 23.

A convolutional block in code

RGB input: Conv 3×3, eight output channels, ReLU, then max pooling.

PyTorch: NCHW

```
from torch import nn
block = nn.Sequential(
    nn.Conv2d(3, 8, 3,
              stride=1,
              padding=1,
              bias=True),
    nn.ReLU(),
    nn.MaxPool2d(2, stride=2,
                 padding=0)
)
```

Keras: NHWC, TensorFlow backend

```
import keras
L = keras.layers
block = keras.Sequential([
    keras.Input((32, 32, 3)),
    L.Conv2D(8, 3, strides=1,
      padding="same", use_bias=True,
      data_format="channels_last"),
    L.ReLU(),
    L.MaxPooling2D(2, strides=2,
      padding="valid",
      data_format="channels_last")
])
```

For this odd kernel and stride 1, Keras `same` equals one zero on each side. These blocks specify the same architecture; matching numerical outputs also requires matching weights (Appendix 25). PyTorch; Keras.

Exercise: an RGB image batch

Work in pairs for 3 minutes, then compare your reasoning.

Batch of two RGB images, each 32×32:

$$\text{Conv } (3 \to 8,\ 3 \times 3,\ s = 1,\ p = 1,\ \text{bias}) \longrightarrow \text{ReLU} \longrightarrow \text{MaxPool } (2 \times 2,\ s = 2).$$

1. Give the input, preactivation, activation and pooled shapes in PyTorch.
2. Write the same four shapes in Keras with `channels_last`.
3. Give the kernel and bias shapes, and count trainable parameters.
4. Find the final receptive field and output spacing in input pixels.

Explain your count: Which quantities depend on batch size or resolution?

Convolution: `groups=1`, dilation 1, one zero on all four sides. Pooling: no padding, floor sizing. ReLU is pointwise. No classification head is included.

Solution: shapes and parameters

Stage	PyTorch: NCHW	Keras: NHWC	Params
Input X	$(2, 3, 32, 32)$	$(2, 32, 32, 3)$	–
Conv output Z	$(2, 8, 32, 32)$	$(2, 32, 32, 8)$	224
ReLU output H	$(2, 8, 32, 32)$	$(2, 32, 32, 8)$	0
Pool output P	$(2, 8, 16, 16)$	$(2, 16, 16, 8)$	0

Kernel: PyTorch $(8, 3, 3, 3)$; Keras $(3, 3, 3, 8)$. Bias: $(8,)$.

$$\boxed{8(3 \cdot 3 \cdot 3 + 1) = 224 \text{ trainable parameters.}}$$

Spatial sizes: $(32 + 2 - 3)/1 + 1 = 32$; $\quad (32 - 2)/2 + 1 = 16$.

Original exercise solution. Parameter count is independent of the two images and the 32×32 resolution. The same learned weights are reused for all windows and all batch elements.

Solution: the input region

For the Conv3 – ReLU – Pool2 block:

$$(r, j): \quad (1,1) \longrightarrow (3,1) \longrightarrow (3,1) \longrightarrow \boxed{(4,2)}.$$

Output position along one axis	Potential input indices
General pooled index i	$2i-1,\ 2i,\ 2i+1,\ 2i+2$
Interior example $i = 1$	$1, 2, 3, 4$: four real positions
Boundary example $i = 0$	$-1, 0, 1, 2$: only $0, 1, 2$ are real

An interior pooled unit has a nominal 4×4 receptive field. At the top-left corner, only 3×3 real input pixels are available.

Apply the index calculation independently to rows and columns, across all three RGB channels. This is potential dependency; zero weights, ReLU and the selected maximum can reduce actual influence. Padding adds constants, not image information.

Session checklist and next steps

1. **Channels:** Does one ordinary filter read one input channel or all of them?
2. **Parameters:** Does doubling image width double the learned weights?
3. **Pooling:** Which dimensions change, and what information can disappear?
4. **Receptive field:** Why does Conv3 then Pool2 give 4×4?

Bridge to Enrique's block

Use shapes, channels and receptive fields to read deep CNNs for images and video.

Next foundations session

Markov assumptions and sequence probabilities: the mathematical bridge to n-grams.

Before leaving: justify one answer using a shape, a formula, or a small window. Appendices 21–26 extend the reference material and are not extra scheduled content.

The full Conv2D operator

Reference | zero-based indices, ordinary channel mixing (`groups=1`).

$$Z_{n,o,i,j} = b_o + \sum_{c=0}^{C_{\text{in}}-1} \sum_{u=0}^{k_H-1} \sum_{v=0}^{k_W-1} W_{o,c,u,v}\, \overline{X}_{n,c,\, is_H - p_T + ud_H,\, js_W - p_L + vd_W}.$$

$\overline{X}$ equals X on the image and zero outside it.

n: batch element

o, c: output and input channels

i, j: output coordinates

u, v: kernel offsets

s_H, s_W: strides

d_H, d_W: kernel dilation factors

$$k_H^{\text{eff}} = d_H(k_H - 1) + 1, \qquad H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} + p_T + p_B - k_H^{\text{eff}}}{s_H} \right\rfloor + 1.$$

The width formula is analogous. The bias is added once per output value.

Count: $C_{\text{out}}(C_{\text{in}} k_H k_W + 1)$. Dilation changes the sampled positions, but does not add coefficients to the kernel.

PyTorch Conv2d; GBC, §9.5. Main slides use $d_H = d_W = 1$.

Pooling at image boundaries

Reference | equal output shapes need not imply equal values.

Average pool $\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$, window 3×3, stride 1, one implicit padding position on every side:

Configuration	Every output entry
PyTorch `AvgPool2d(3, stride=1, padding=1)`	$10/9$
PyTorch with `count_include_pad=False`	$10/4 = 2.5$
TensorFlow / Keras–TensorFlow, `padding="SAME"`	$10/4 = 2.5$

- Max pooling excludes padding from the maximum (conceptually $-\infty$). Manually padding a negative image with zeros can change the result.
- TensorFlow `SAME`: output size $\lceil n/s \rceil$; padding may be asymmetric.
- PyTorch `ceil_mode=True` and TensorFlow `SAME` are different settings.

PyTorch AvgPool2d; MaxPool2d; TensorFlow avg_pool2d; Keras TensorFlow backend. Main pooling examples use `VALID` with no padding.

Receptive field, jump and alignment

Reference | use the following recurrence independently on each spatial axis.

$$k_\ell^{\mathrm{eff}} = d_\ell(k_\ell - 1) + 1, \quad r_\ell = r_{\ell-1} + (k_\ell^{\mathrm{eff}} - 1)j_{\ell-1}, \quad j_\ell = s_\ell j_{\ell-1}.$$

Let a_ℓ be the center of output index zero in input coordinates:

$$a_\ell = a_{\ell-1} + \left(\frac{k_\ell^{\mathrm{eff}} - 1}{2} - p_\ell^-\right) j_{\ell-1}, \qquad r_0 = j_0 = 1, \quad a_0 = \tfrac{1}{2}.$$

Operation	r	j	a
Conv 3, stride 1, padding 1	3	1	0.5
Pool 2, stride 2, no padding	4	2	1
Optional further Conv 3, stride 1, padding 1	8	2	1

Output index t has center $a_\ell + tj_\ell$; p_ℓ^- is padding before the input. Nominal spans can include padded positions or gaps; actual influence also depends on values.

Recurrences and localization adapted from Araujo, Norris & Sim, Computing Receptive Fields of Convolutional Neural Networks (2019). Pixel centers are at 0.5, 1.5, 2.5, ... The further convolution is an optional extension, outside the main exercise.

Gradients through pooling

Reference | connect local pooling to the chain rule.

$$A = \begin{bmatrix} 2 & 1 \\ 1 & 0 \end{bmatrix}, \qquad p_{\max} = 2, \quad p_{\text{avg}} = 1, \qquad \frac{\partial \mathcal{L}}{\partial p} = 6.$$

Max pooling

The maximum is unique, at the top left.

$$\frac{\partial \mathcal{L}}{\partial A} = \begin{bmatrix} 6 & 0 \\ 0 & 0 \end{bmatrix}.$$

The derivative goes to the selected entry.

Average pooling

Each entry has derivative $1/4$.

$$\frac{\partial \mathcal{L}}{\partial A} = \begin{bmatrix} 1.5 & 1.5 \\ 1.5 & 1.5 \end{bmatrix}.$$

All four entries receive a contribution.

For overlapping windows, contributions add. At tied maxima the max function is nondifferentiable; the implementation selects a backward rule.

Original chain-rule calculation, no padding. A is the first 2×2 patch of H on slide 4. No pooling weights are learned, but gradients still propagate to earlier trainable layers. GBC, §9.3 and §6.5.

Matching weights across libraries

Reference | arrays X, W and b use NCHW, OIHW and $(C_{\text{out}},)$.

```
# NumPy: change axes; do not reverse the spatial kernel.
x = X.transpose(0, 2, 3, 1)    # NCHW -> NHWC
w = W.transpose(2, 3, 1, 0)    # OIHW -> HWIO
```

Keras 3, TensorFlow backend

```
layer = keras.layers.Conv2D(
    2, 2, strides=1, padding="valid",
    use_bias=True, activation=None,
    data_format="channels_last",
    dtype="float64")
layer.build(x.shape)
layer.set_weights([w, b])
z_keras = layer(x).numpy()
```

TensorFlow: an operation

```
z_tf = tf.nn.conv2d(
    x, w, strides=(1, 1),
    padding="VALID",
    data_format="NHWC")
z_tf = tf.nn.bias_add(
    z_tf, b, data_format="NHWC")
z_nchw = z_tf.numpy().transpose(
    0, 3, 1, 2)
```

Use the same values, dtype, padding and activation to compare outputs. The two-channel example has $Z_{0,0,0,1} = 4$ in NCHW.

Imports and all arrays are in `session8_frameworks.py`. Keras Conv2D; TensorFlow conv2d. The low-level TensorFlow operation receives a filter tensor; it does not create trainable weights or add bias automatically.

Reading and reproducible examples

Concepts and mathematical explanations

- Goodfellow, Bengio & Courville (2016), *Deep Learning*, Ch. 9: §9.1 convolution, §9.2 motivation, §9.3 pooling, §9.5 variants.
- Christopher M. Bishop & Hugh Bishop (2024), *Deep Learning: Foundations and Concepts*, Ch. 10, Convolutional Networks.
- Araujo, Norris & Sim (2019), Computing Receptive Fields of Convolutional Neural Networks, *Distill*.

Translate the notation into an API

PyTorch Conv2d, MaxPool2d, AvgPool2d; Keras Conv2D and pooling layers; TensorFlow conv2d.

Companion ZIP: examples, arrays and instructions

```
python session8_numeric.py
python session8_frameworks.py --backend numpy
python session8_frameworks.py --backend torch       # optional
python session8_frameworks.py --backend tensorflow  # optional
```

Original example programs. Official API documentation consulted 18 October 2026.